

Binding Views and Handling Actions

Contents

- **Binding views**
- **Handling actions**
- **Common event listeners**

Binding View

- Assign id for view in xml file

```
<Button  
    android:id="@+id/button"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_below="@+id/textView"  
    android:layout_centerHorizontal="true"  
    android:layout_marginTop="48dp"  
    android:text="Login" />
```

Binding View

- Access view through `findViewById()`

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    //Bind layout
    setContentView(R.layout.activity_login);

    //Bind view
    Button button = (Button) findViewById(R.id.button);

    //Set event on click for button
    button.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {

            // todo something

        }
    });
}
```

Handling Action

- **An UI event is an action that is triggered by the user interacting with the UI**
 - A button pressed or released
 - A key is pressed or released
 - An area of a touch screen is touched, etc.
 - Gesture
 - long touch
 - double-tap
 - Fling
 - Drag
 - Scroll
 - pinch

Handling Action – 1st way

- Set event name in xml file: android:onClick="EventName"

```
<Button  
    android:id="@+id/buttonLogin"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:onClick="onLogin"  
    android:text="Login" />
```

Handling Action – 1st way

- In activity, implement event

```
public class MainActivity extends AppCompatActivity {  
    Button btnLogin;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        btnLogin = (Button)findViewById(R.id.button_login);  
    }  
  
    public void onLogin(View view){  
        Toast.makeText(MainActivity.this, "Bạn vừa đăng nhập", Toast.LENGTH_SHORT).show();  
    }  
}
```

Handling Action – 2nd way

- Inline anonymous listener

```
public class MainActivity2 extends AppCompatActivity {
    Button btnLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main_2);

        btnLogin = (Button)findViewById(R.id.button_login);

        btnLogin.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Toast.makeText(MainActivity2.this, "Bạn vừa đăng nhập!", Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```


Handling Action – 3th way

- Activity implements View.OnClickListener

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        findViewById(R.id.btn_login).setOnClickListener(this);
        findViewById(R.id.btn_register).setOnClickListener(this);
        findViewById(R.id.tv_forgot_pass).setOnClickListener(this);
    }

    @Override
    public void onClick(View view) {
        if (view.getId() == R.id.btn_login){
            //TODO:
        } else if (view.getId() == R.id.btn_register){
            //TODO:
        } else if (view.getId() == R.id.tv_forgot_pass){
            //TODO:
        }
    }
}
```

View is base class for input controls

- The [View](#) class is the basic building block for all UI components, including input controls
- View is the base class for classes that provide interactive UI components
- View provides basic interaction through `android.onClick`

View Event listeners

- `setOnClickListener` - Callback when the view is clicked
- `setOnDragListener` - Callback when the view is dragged
- `setOnFocusChangeListener` - Callback when the view changes focus
- `setOnGenericMotionListener` - Callback for arbitrary gestures
- `setOnHoverListener` - Callback for hovering over the view
- `setOnKeyListener` - Callback for pressing a hardware key when view has focus
- `setOnLongClickListener` - Callback for pressing and holding a view
- `setOnTouchListener` - Callback for touching down or up on a view

AdapterView Event Listeners

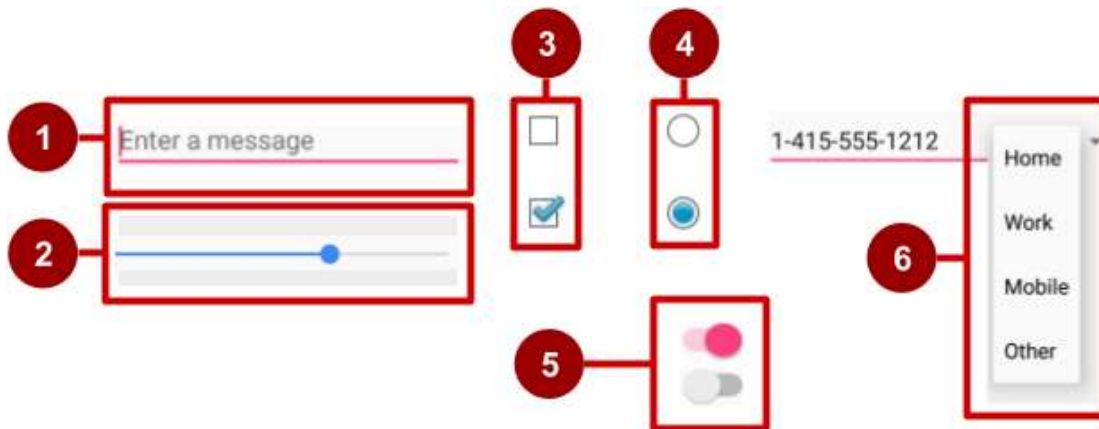
- `setOnItemClickListener` - Callback when an item contained is clicked
- `setOnItemLongClickListener` - Callback when an item contained is clicked and held
- `setOnItemSelectedListener` - Callback when an item is selected

EditText Common Listeners

- `addTextChangedListener` - Fires each time the text in the field is being changed
- `setOnEditorActionListener` - Fires when an "action" button on the soft keyboard is pressed

Examples of input controls

- [EditText](#)
- [SeekBar](#)
- [CheckBox](#)
- [RadioButton](#)
- [Switch](#)
- [Spinner](#)



How input controls work

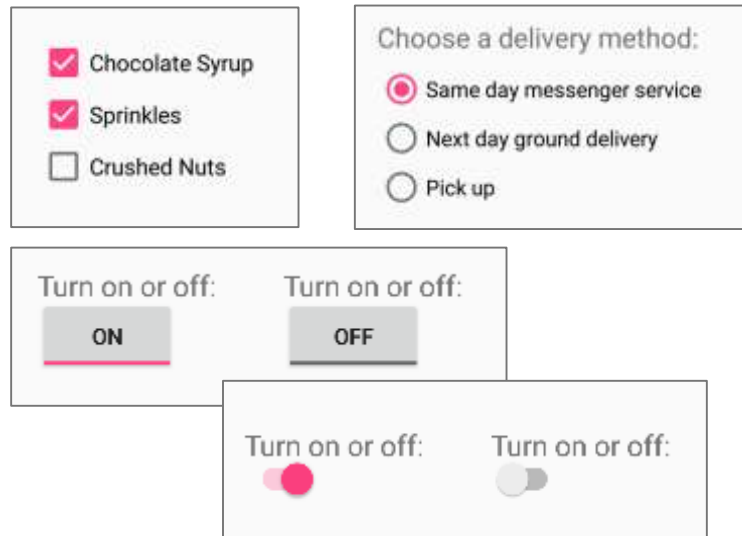
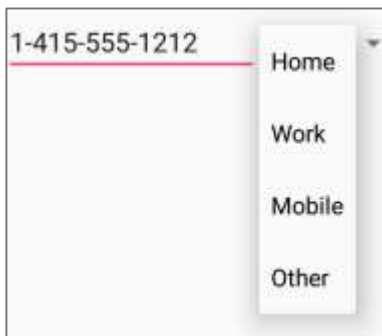
1. Use EditText for entering text using keyboard
2. Use SeekBar for sliding left or right to a setting
3. Combine CheckBox elements for choosing more than one option
4. Combine RadioButton elements into [RadioGroup](#) — user makes only one choice
5. Use Switch for tapping on or off
6. Use Spinner for choosing a single item from a list

UI elements for providing choices

- CheckBox and RadioButton

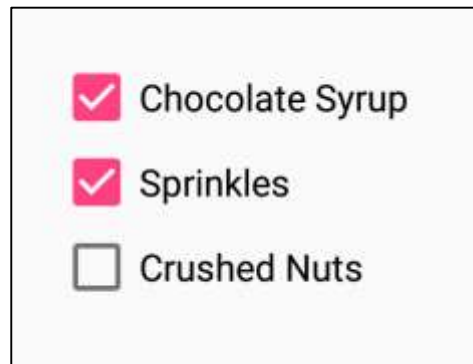
- ToggleButton and Switch

- Spinner



CheckBox

- User can select any number of choices
- Checking one box does not uncheck another
- Users expect checkboxes in a vertical list
- Commonly used with a Submit button
- Every CheckBox is a View and can have an onClick handler



☒ Chocolate Syrup

☒ Sprinkles

☐ Crushed Nuts

RadioButton

- Put [RadioButton](#) elements in a [RadioGroup](#) in a vertical list (horizontally if labels are short)
- User can select only one of the choices
- Checking one unchecks all others in group
- Each [RadioButton](#) can have onClick handler
- Commonly used with a Submit button for the RadioGroup

Choose a delivery method:

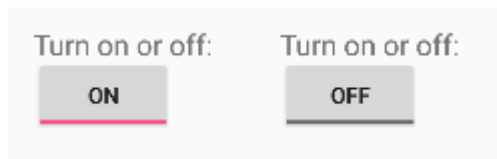
☒ Same day messenger service

☐ Next day ground delivery

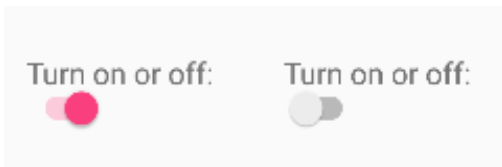
☐ Pick up

Toggle buttons and switches

- User can switch between on and off
- Use `android:onClick` for click handler



Toggle buttons



Switches

Classwork

- **On Login screen, when click Login button:**
 - If username is admin and password is 123456 => show “Login successfully”
 - Otherwise, show “Login failed”

References

- <https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/>